

Gün 16 — Sürükle-Bırak Yükleme Arayüzü

Hedef

Kullanıcının şüpheli bir görsel dosyasını (PNG/JPEG) yükleyebileceği basit, bağımsız bir web arayüzü (`frontend/`) oluşturmak: sürükle-bırak (drag & drop) alanı, tıklayarak dosya seçme, ve yükleme sırasında gösterilecek bir loading göstergesi. Backend'e (`/api/v1/analyze`) gerçek `fetch` entegrasyonu bilinçli olarak kapsam dışı bırakıldı — plana göre bu Gün 17'nin konusu.

Yaklaşım

1. Dosya yapısı

`frontend/` altında üç bağımsız dosya oluşturuldu (framework/build adımı yok, doğrudan tarayıcıda `index.html` açılarak çalışır): - `index.html` — sayfa iskeleti (dropzone, gizli `<input type="file">`, hata mesajı alanı, dosya önizleme kartı, loading göstergesi, sonuç bölümü için boş bir `#results` konteyneri — bu Gün 17'de doldurulacak). - `style.css` — koyu temalı, bağımsız (framework'süz) stil; responsive ve tek sütunlu (`max-width: 640px`) bir düzen. - `app.js` — IIFE içinde, global scope'u kirletmeyen vanilla JS.

2. Sürükle-bırak implementasyonu

`dropzone` elemanına şu event'ler bağlandı: - `click` → gizli `#file-input` 'i tetikler (klasik "tıkla ve seç" akışı). - `keydown` (Enter/Space) → erişilebilirlik için aynı davranış (`tabindex="0"` , `role="button"` , `aria-label` ile klavye/screen-reader kullanıcıları destekleniyor). - `dragover` → `event.preventDefault()` (tarayıcının varsayılan "dosyayı yeni sekmede aç" davranışını engeller) + `dropzone--dragover` sınıfı ile görsel geri bildirim (kenarlık/arka plan rengi değişir). - `dragleave` → görsel geri bildirimin kaldırılması. - `drop` → `event.preventDefault()` + `event.dataTransfer.files[0]` alınıp `handleFile()` 'a iletilir. - `#file-input` 'in `change` event'i de aynı `handleFile()` 'ı çağırır — böylece hem sürükle-bırak hem klasik dosya seçici tek bir kod yolundan geçer.

3. İstemci tarafı doğrulama

Backend'deki (`backend/app/main.py`) kurallarla bilinçli olarak birebir aynı sınırlar istemci tarafında da uygulandı (kullanıcıya anında geri bildirim vermek için — backend doğrulaması hâlâ tek gerçek

kaynak, bu yalnızca UX katmanı): - Yalnızca `image/png` / `image/jpeg` (`file.type` kontrolü). - Boş (0 bayt) dosya reddi. - 25 MB üst sınır (`MAX_UPLOAD_SIZE` , backend'deki değerle aynı).

Doğrulama hatası `#error-message` alanında Türkçe, anlamlı bir mesajla gösteriliyor; geçerli bir dosya seçildiğinde ise önceki hata otomatik temizleniyor.

4. Dosya önizleme ve loading göstergesi

- Geçerli bir dosya seçildiğinde `URL.createObjectURL()` ile küçük bir thumbnail, dosya adı ve biçimlendirilmiş boyutu (`formatFileSize` — B/KB/MB/GB) gösteren bir kart (`#file-preview`) beliriyor; "x" butonuyla seçim geri alınabiliyor (`URL.revokeObjectURL()` ile bellek sızıntısı önleniyor).
- `#loading` (spinner + "Analiz ediliyor..." metni) `setLoading(bool)` fonksiyonuyla açılıp kapatılabilecek şekilde hazırlandı; Gün 16 kapsamında henüz gerçek bir ağ isteği tetiklenmediği için bu gösterge şu an yalnızca altyapı olarak duruyor — Gün 17'de `fetch` çağrısının `try/finally` 'sinde kullanılacak.

Dosyalar

```
frontend/index.html # sayfa iskeleti: dropzone, dosya input, önizleme, loading, results
frontend/style.css  # koyu tema, dropzone/loading/preview stilleri
frontend/app.js      # drag&drop, tıkla-seç, istemci doğrulaması, önizleme, loading toggle
```

Test Sonuçları

Bu oturumda tarayıcı otomasyonu (Claude in Chrome) bağlı değildi, bu yüzden gerçek bir tarayıcıda sürükle-bırak etkileşimi otomatik olarak doğrulanamadı — bu açıkça not ediliyor. Bunun yerine:

- `python3 -m http.server` ile `frontend/` dizini statik olarak sunuldu; `index.html` , `style.css` , `app.js` için `curl` ile 200 OK doğrulandı (dosyaların birbirine doğru yollarla bağlandığı teyit edildi).
- `app.js` elle gözden geçirildi (proje ortamında `node` kurulu olmadığı için otomatik sözdizimi kontrolü yapılamadı); tüm `document.getElementById` çağrılarının `index.html` 'deki id'lerle (`dropzone` , `file-input` , `error-message` , `file-preview` , `file-thumbnail` , `file-name` , `file-size` , `remove-file` , `loading` , `results`) birebir eşleştiği satır satır kontrol edildi.
- Kullanıcının kendi tarayıcısında `frontend/index.html` 'i doğrudan açıp (`file://` ile de çalışır, harici bir kaynağa bağımlılık yok) bir PNG/JPEG dosyasını sürükleyip bırakarak veya

tıklayarak seçmesi önerilir — bu, Gün 16'nın kabul kriterinin (arayüzün tarayıcıda açılıp bir dosyanın sürükle-bırak ile seçilebilmesi) nihai doğrulamasıdır.

Kabul Kriterleri — Durum

- [x] Arayüz tarayıcıda açılabilir durumda (`frontend/index.html` , statik dosya sunumuyla test edildi; harici bağımlılık yok)
- [x] Sürükle-bırak alanı implementasyonu tamamlandı (kod incelemesiyle doğrulandı); **kullanıcının kendi tarayıcısında elle son doğrulaması önerilir** çünkü bu oturumda tarayıcı otomasyonu bağlı değildi

Notlar / Riskler

- `frontend/` şu an backend'den (`http://127.0.0.1:8000`) bağımsız statik dosyalar olarak duruyor; Gün 17'de `fetch('/api/v1/analyze', ...)` eklenince farklı origin'lerden servis edilme ihtimaline karşı FastAPI tarafında `CORSMiddleware` yapılandırması gerekecek (plana zaten Gün 17 alt görevi olarak yazılı).
- İstemci tarafı doğrulama (tip/boyut) yalnızca UX amaçlıdır; gerçek güvenlik sınırı backend'de (`backend/app/main.py` — magic bytes doğrulaması dahil) kalmaya devam ediyor. İstemci kontrolleri atlatılabilir olduğundan tek başına güvenilir kabul edilmemeli.